

COURSE CODE/TITLE:	CPE 010 Data Structures and Algorithms	SECTION:	CPE21S2
DATE PERFORMED:	07/17/2026	DATE SUBMITTED:	07/17/2026
NAME:	Jiro Luis A. Katindig	INSTRUCTOR:	Engr. Jimlord M. Quejado

ACTIVITY NO. <u>2</u>	Arrays, Pointers and Dynamic Memory Allocation
1. PROCEDURE OUTPUT	

Table 2-1. Initial Driver Program

Screenshot	<pre> 1 #include <iostream> 2 #include <string.h> 3 4 class Student{ 5 6 private: 7 std::string studentName; 8 int studentAge; 9 10 public: 11 12 //constructor 13 Student(std::string newName = "John Doe", int newAge=18){ 14 studentName = std::move(newName); 15 studentAge = newAge; 16 std::cout << "Constructor Called." << std::endl; 17 }; 18 19 //destructor 20 ~Student(){ 21 std::cout << "Destructor Called." << std::endl; 22 } 23 24 //Copy Constructor 25 Student(const Student &copyStudent){ 26 std::cout << "Copy Constructor Called" << std::endl; 27 studentName = copyStudent.studentName; 28 studentAge = copyStudent.studentAge; 29 } 30 31 //Display Attributes 32 void printDetails(){ 33 std::cout << this->studentName << " " << this->studentAge << std::endl; 34 } 35 }; 36 37 int main() { 38 Student student1("Roman", 28); 39 Student student2(student1); 40 Student student3; 41 student3 = student2; 42 return 0; 43 } </pre> <pre> Constructor Called. Copy Constructor Called Constructor Called. Destructor Called. Destructor Called. Destructor Called. </pre>
Observation	I've observed that the program successfully demonstrates how the constructors, copy constructors, and destructor works. When I created the "Student" object, the constructor was called automatically. When I copied an existing object (copied student 1 in student 2) in the int, copy constructor was called instead. And when the program ended "return 0", the destructor was called for each object, showing each was deleted properly.

Table 2-2. Modified Driver Program with Student Lists

Screenshot	<pre style="background-color: #2e3436; color: #eeeeec; padding: 10px; width: 60%;"> int main() { const size_t j = 5; Student studentList[j] = {}; std::string namesList[j] = {"Den", "Ann", "Chelle", "Jiro", "Luis"}; int ageList[j] = {15, 16, 18, 19, 16}; return 0; } </pre> Constructor Called. Constructor Called. Constructor Called. Constructor Called. Constructor Called. Destructor Called. Destructor Called. Destructor Called. Destructor Called. Destructor Called.
Observation	So when I modified the program, and make it create an array that consists of 5 “student” objects, constructor was automatically called based on the output 5 times, because I’ve made the size to 5 and named 5 objects in the list. And when the program was finished, destructor was called 5 times again, showing each was destroyed successfully. With this in mind, I properly understand how constructors and destructors work when working with arrays.

Table 2-3. Final Driver Program

Screenshot	<pre>int main() { const size_t j = 5; Student studentList[j] = {}; std::string namesList[j] = {"Den", "Ann", "Chelle", "Jiro", "Luis"}; int ageList[j] = {15, 16, 18, 19, 16}; for(int i = 0; i < j; i++){ //loop A Student *ptr = new Student(namesList[i], ageList[i]); studentList[i] = *ptr; } for(int i = 0; i < j; i++){ //loop B studentList[i].printDetails(); } return 0; }</pre>	
Loop A	<pre>for(int i = 0; i < j; i++){ //loop A Student *ptr = new Student(namesList[i], ageList[i]); studentList[i] = *ptr; }</pre>	
Observation	So in loop A, I've observed that it creates a new "student" object for each name and age in the arrays with the help of ptr and address we've learned. It stores the object in the "studentList" array. This is just setting up the "student" objects so that when they are called in the future, it would show the name and age.	
Loop B	<pre>for(int i = 0; i < j; i++){ //loop B studentList[i].printDetails(); } return 0;</pre>	
Observation	I've observed in Loop B, it goes through each "student" object in the array and calls the "printDetails()". After that, it will display the name and age of every student created in the Loop A.	
Output	<pre>Constructor Called. Constructor Called. Constructor Called. Constructor Called. Constructor Called. Constructor Called. Constructor Called. Constructor Called. Constructor Called. Constructor Called. Den 15 Ann 16 Chelle 18 Jiro 19 Luis 16 Destructor Called. Destructor Called. Destructor Called. Destructor Called. Destructor Called.</pre>	
Observation	So here, I can see that 10 constructor were called, 5 are when creating the students on the very start, then the other 5 are when we tried to create another set of 5 students using ptr, and after that, the names and age were displayed with the help of the "printDetails", and at last is when the program was finished again. 5 destructors were called because there were a memory leak, the ones that were created in the loop A was never destroyed, thus I'll have to modify it.	

Table 2-4. Modifications/Corrections Necessary

Modifications	<pre>for(int i = 0; i < j; i++){ //loop A Student *ptr = new Student(namesList[i], ageList[i]); studentList[i] = *ptr; delete ptr; // modified for destructor/deleting loop A }</pre> Constructor Called. Destructor Called. Constructor Called. Destructor Called. Constructor Called. Destructor Called. Constructor Called. Destructor Called. Constructor Called. Destructor Called.
Observations	So after adding “delete ptr”, I’ve observed that destructor were immediately called after every new “student” ptr object were made in loop A, which resulted to the constructor and destructor being called alternatively during the loop. This shows that the object were properly deleted and memory were released, which made the programs management of the memory better as it prevented memory leaks.
Final Modification	<pre>//Copy Assignment Operator Student& operator=(const Student& copyStudent) { std::cout << "Copy Assignment Operator Called." << std::endl; if (this != &copyStudent) { // Prevent self-assignment studentName = copyStudent.studentName; studentAge = copyStudent.studentAge; } return *this; };</pre> Constructor Called. Copy Assignment Operator Called. Destructor Called. Constructor Called. Copy Assignment Operator Called. Destructor Called. Constructor Called. Copy Assignment Operator Called. Destructor Called. Constructor Called. Copy Assignment Operator Called. Destructor Called. Constructor Called. Copy Assignment Operator Called. Destructor Called.
Observation	So since I wasn’t able to add the copy assignment operator earlier, I just added it now. After adding it, copy assignment operator were called every time there’s an existing “student” object that its values were assigned to another object. It creates a temporary object first, then the copy assignment operator copied its data to the corresponding object in the “studentList”, and since delete ptr was added, it deletes it immediately.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

Code

```
1 #include <iostream>
2 #include <string>
3
4 // Base Class to allow storing both Fruit and Vegetable in one array
5 class Groceries {
6 protected:
7     std::string type;
8     std::string name;
9     double price;
10    int quantity;
11
12 public:
13     // Constructor
14     Groceries(std::string T = "", std::string N = "", double P = 0.0, int Q = 0)
15         : type(T), name(N), price(P), quantity(Q) {}
16
17     // Virtual Destructor (essential for base classes with virtual methods)
18     virtual ~Groceries() {
19         std::cout << name << " has been bagged." << std::endl;
20     }
21
22     // Copy Constructor
23     Groceries(const Groceries &copyGroceries)
24         : type(copyGroceries.type), name(copyGroceries.name),
25         price(copyGroceries.price), quantity(copyGroceries.quantity) {}
26
27     // Copy Assignment Operator
28     Groceries& operator=(const Groceries& copyGroceries) {
29         if (this != &copyGroceries) {
30             type = copyGroceries.type;
31             name = copyGroceries.name;
32             price = copyGroceries.price;
33             quantity = copyGroceries.quantity;
34         }
35         return *this;
36     }
37
38     // Calculate total price for this item
39     virtual double calculateSum() const {
40         return price * quantity;
41     }
42 }
```

```
43 // Display item details
44 virtual void ScannedItems() const {
45     std::cout << "Type: " << type << std::endl;
46     std::cout << "Name: " << name << std::endl;
47     std::cout << "Price: PHP " << price << std::endl;
48     std::cout << "Quantity: " << quantity << std::endl;
49     std::cout << "Total: PHP " << calculateSum() << std::endl;
50     std::cout << std::endl;
51 }
52 };
53
54 // Problem 1: Fruit Class
55 class Fruit : public Groceries {
56 public:
57     // Constructor
58     Fruit(std::string name = "", double price = 0.0, int quantity = 0)
59         : Groceries("Fruit", name, price, quantity) {}
60
61     // Destructor
62     ~Fruit() override {}
63
64     // Copy Constructor
65     Fruit(const Fruit &copyFruit) : Groceries(copyFruit) {}
66
67     // Copy Assignment Operator
68     Fruit& operator=(const Fruit& copyFruit) {
69         if (this != &copyFruit) {
70             Groceries::operator=(copyFruit);
71         }
72         return *this;
73     }
74 };
```

```
76 // Problem 1: Vegetable Class
77 class Vegetable : public Groceries {
78 public:
79     // Constructor
80     Vegetable(std::string name = "", double price = 0.0, int quantity = 0)
81         : Groceries("Vegetable", name, price, quantity) {}
82
83     // Destructor
84     ~Vegetable() override {}
85
86     // Copy Constructor
87     Vegetable(const Vegetable &copyVegetable) : Groceries(copyVegetable) {}
88
89     // Copy Assignment Operator
90     Vegetable& operator=(const Vegetable& copyVegetable) {
91         if (this != &copyVegetable) {
92             Groceries::operator=(copyVegetable);
93         }
94         return *this;
95     }
96 };
97
98 // Problem 3: Calculate sum of all objects in Jenna's Grocery List
99 double TotalSum(Groceries* GroceryList[], int groceries) {
100     double total = 0.0;
101
102     for (int i = 0; i < groceries; i++) {
103         if (GroceryList[i] != nullptr) {
104             total += GroceryList[i]->calculateSum();
105         }
106     }
107     return total;
108 }
```

```
110 int main() {
111     // Problem 2: GroceryList array storing instances
112     Groceries* GroceryList[4];
113
114     GroceryList[0] = new Fruit("Apple", 10.0, 7);
115     GroceryList[1] = new Fruit("Banana", 10.0, 8);
116     GroceryList[2] = new Vegetable("Broccoli", 60.0, 12);
117     GroceryList[3] = new Vegetable("Lettuce", 50.0, 10);
118
119     std::cout << "~~~~~Jiro's Grocery List~~~~~\n\n";
120
121     // Display all items
122     for (int i = 0; i < 4; i++) {
123         if (GroceryList[i] != nullptr) {
124             GroceryList[i]->ScannedItems();
125         }
126     }
127
128     std::cout << "Total Cost: PHP " << TotalSum(GroceryList, 4) << std::endl;
129
130     // Problem 4: Delete Lettuce and de-allocate memory
131     delete GroceryList[3];
132     GroceryList[3] = nullptr; // Prevents dangling pointer
133
134     std::cout << "\nAfter deleting Lettuce:" << std::endl;
135     std::cout << "Updated Total Cost: PHP " << TotalSum(GroceryList, 4) << std::endl;
136
137     // Clean up remaining allocated memory
138     for (int i = 0; i < 3; i++) {
139         if (GroceryList[i] != nullptr) {
140             delete GroceryList[i];
141             GroceryList[i] = nullptr;
142         }
143     }
144
145     return 0;
146 }
```

Output

```
~~~~~Jiro's Grocery List~~~~~  
  
Type: Fruit  
Name: Apple  
Price: PHP 10  
Quantity: 7  
Total: PHP 70  
  
Type: Fruit  
Name: Banana  
Price: PHP 10  
Quantity: 8  
Total: PHP 80  
  
Type: Vegetable  
Name: Broccoli  
Price: PHP 60  
Quantity: 12  
Total: PHP 720  
  
Type: Vegetable  
Name: Lettuce  
Price: PHP 50  
Quantity: 10  
Total: PHP 500  
  
Total Cost: PHP 1370  
Lettuce has been bagged.  
  
After deleting Lettuce:  
Updated Total Cost: PHP 870  
Apple has been bagged.  
Banana has been bagged.  
Broccoli has been bagged.
```

1. Summary of Lessons Learned

In this activity, I really got to refresh my understanding of how C++ handles object lifecycles like constructors, copy constructors, copy assignment operators, and destructors. I got hands-on experience using `new` and `delete` to allocate memory on the heap, and I saw firsthand how forgetting to free memory causes memory leaks. I also learned how to use object-oriented programming concepts like inheritance and virtual functions, which allowed me to store different derived objects inside a single array of base class pointers.

2. Analysis of the Procedure

The procedure was pretty straightforward and easy to follow because it built things up step by step. At first, I observed how stack-allocated objects automatically trigger constructors and destructors on their own. But when I moved to Loop A with arrays and dynamic allocation, I noticed a memory leak happening because the heap objects created with `new` were never deleted. Once I updated the code by adding `delete ptr` and setting up the copy assignment operator, I could see C++ properly handle the copying and cleanup, which completely fixed the memory leak.

3. Analysis of the Supplementary Activity

For the supplementary activity, I applied these concepts to build Jenna's grocery list. I created a base `Groceries` class along with `Fruit` and `Vegetable` derived classes. Storing these in an array of pointers made it easy to loop through, display their details, and calculate the initial total of PHP 1,370 using the `TotalSum` function. After that, I deleted the Lettuce object from memory and set its pointer to `nullptr` to avoid dangling pointers, which updated the total cost to PHP 870.

4. Concluding Statement / Feedback

I think I did really well in this activity since I was able to spot the memory leak in the procedure, fix it properly, and get all the correct outputs for the supplementary activity. Walking through the code line by line helped me see how object lifecycles actually work under the hood. For my areas of improvement, I want to practice using smart pointers like `std::weak\_ptr` so I do not have to manually delete memory, and I also want to look into move semantics to optimize copying.

Tools Used:

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use:

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:



Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.